

动态规划入门

• 我该怎么看出这是一道DP

- 求最大/小值求方案数量求子序列
- 贪心做不了(有反例)
- 依赖前面的值，无后效性

• 怎么写DP

- 状态是谁？状态有哪些？状态必须包含所有能够影响下一阶段决策的信息
- 怎么转移？dp[i]是从哪里转移过来的？怎么转移代价最小/收益最大/能计算全部路径？
- 初始状态

• DP数组怎么写

- 一般求什么什么就是DP数组
- 要不要开二维？就问自己：“如果有两个人同时走到第 i 步，但他们之前的经历不同，这种不同会限制他们接下来的选择吗？”
- 能不能压缩？用滚动数组来代替二维，我们只考虑会影响结果的值。只记我们需要记的，过期的扔掉。

DP原理

数字三角形

用途：从顶到底或底到顶，求最大路径和。

常见坑：从底到顶递推可省去边界处理。

```
void solve() {
    int n;
    cin >> n;
    int sjx[100][100];
    for (int i = 0; i < n; i++)
        for (int j = 0; j <= i; j++)
            cin >> sjx[i][j];
    for (int i = n-2; i >= 0; i--)
        for (int j = 0; j <= i; j++)
            sjx[i][j] += max(sjx[i+1][j], sjx[i+1][j+1]);
    cout << sjx[0][0] << '\n';
}
```

复杂度： $O(n^2)$ ，空间 $O(1)$ （直接在输入数组上修改）。

2.5 记忆化搜索

用途：从未知到已知递推，适合复杂状态转移。

与 DP 区别：

- DP: 从已知到未知, 严格顺序, 循环实现, 常数小。
- 记忆化搜索: 从未知到已知, 递归实现, 需状态数组防重复计算。

```
vector<int> mama(100230, -1);
int dfs(int a) {
    if (a == 1) return 1;
    if (mama[a] != -1) return mama[a];
    int cnt = 1;
    for (int i = 1; i <= a / 2; i++)
        cnt += dfs(i);
    mama[a] = cnt;
    return cnt;
}
void solve() {
    int n;
    cin >> n;
    cout << dfs(n) << '\n';
}
```

复杂度: $O(n^2)$, 空间 $O(n)$ (状态数)。

线性DP

P1874 快速求和

- **核心模型:** 划分段落线性dp->把一个序列切成若干段, 每段满足某种条件, 求最优解
- **思维误区 (Bug):**
- **修正逻辑 (Patch):** 通用思维: $dp[i]$ = 考虑前 i 个元素, 满足条件的最优解; 它是一条线, 从左到右一步步往前延伸。
- **关键代码:**

```
void solve()
{
    string s;
    cin >> s;
    vi nums(s.size());
    for (int i = 0; i < s.size(); i++)
    {
        nums[i] = s[i] - '0';
    }

    int a;
    cin >> a;

    vector<vi> dp(1e5, vi(40, INF));
    dp[0][0] = 0;

    for (int i = 1; i <= s.size(); i++)
```

```

{
    for (int k = 0; k < i; k++)
    {

        bool can = 1;
        int now = 0;
        for (int j = k; j < i; j++)
        {
            now *= 10;
            now += nums[j];

            if (now > a)
            {
                can = 0;
                break;
            }
        }
        if (!can)
        {
            continue;
        }

        for (int val = now; val <= a; val++)
        {
            if (dp[val-now][k] < INF)
            {
                dp[val][i] = min(dp[val][i], dp[val-now][k] + 1);
                // cerr<<now+val<<' ';
            }
        }
    }
}
if (dp[a][s.size()] == INF)
    cout << -1 << '\n';
else
    cout << dp[a][s.size()] - 1 << '\n';
}

```

Blackboard

- **核心模型**:划分型dp, 注意到它的性质是有单调性的; 判断区间是否能成为一个集合当且仅当 `if ((nums[tp] & mask) == 0)`
- **思维误区 (Bug)**:注意如果中间塞很多很多0就会tle->可以选择链表来往上找, 记得要加上区间dp和 (我用树状数组处理, 注意下标)
- **修正逻辑 (Patch)**:
- **关键代码**:

```
struct Fenwick
{
    int n;
    vector<long long> tr;

    // 初始化: 传入最大长度 n
    Fenwick(int size) : n(size), tr(size + 1, 0) {}
    // 初始化2: 节省时间开销的init
    void init(int size)
    {
        n = size;
        tr.assign(n + 1, 0);
    }

    // 核心位运算: 获取二进制最低位的 1
    int lowbit(int x)
    {
        return x & -x;
    }

    // 单点修改: 在位置 x 加上 val
    void add(int x, long long val)
    {
        for (; x <= n; x += lowbit(x))
        {
            tr[x] = (tr[x] + val) % MOD;
        }
    }

    // 查询前缀和: 查询 [1, x] 的和
    long long ask(int x)
    {
        long long res = 0;
        for (; x > 0; x -= lowbit(x))
        {
            res = (res + tr[x]) % MOD;
        }
        return res;
    }

    // 区间查询: 查询 [l, r] 的和
    long long range_ask(int l, int r)
    {
        if (l > r)
            return 0;
        return (ask(r) % MOD - ask(l - 1) % MOD + MOD) % MOD;
    }
};

void solve()
{
    int n;
    cin >> n;
```

```

vi nums(n + 1);
int lst_idx = 0;
vi preidx(n + 1);
rep(i, 1, n)
{
    cin >> nums[i];
    preidx[i] = lst_idx;
    if (nums[i])
        lst_idx = i;
}
// dbg_arr(preidx, 0, preidx.size() - 1);
Fenwick dp(n + 2);
dp.add(1, 1);

for (int i = 1; i <= n; i++)
{
    int mask = 0;
    int tp = i;
    int cnt = 0;
    int now = 0;

    while (tp > 0 && cnt < 32)
    {
        // cerr<<preidx[tp]<< ' ' << cnt << '\n';

        if ((nums[tp] & mask) == 0)
        {
            mask |= nums[tp];

            tp = preidx[tp];
            now = tp ;
        }
        else
        {
            // now=tp+1;
            break;
        }

        cnt++;
    }
    // if (tp == 0)
    //     now = 1;
    dp.add(i + 1, dp.range_ask(now+1, i));
}
cout << dp.range_ask(n + 1, n + 1) << '\n';
}

```

E. LIS of Sequence

- **核心模型**: LIS的性质, 寻找“可能会出现”的小trick (拼接法)

- **思维误区 (Bug):** LIS注意是要求你单调递增还是不上降, 哈希表可能会用重复的数字, 所以我们需要对于每一个数字单独判断
- **修正逻辑 (Patch):** 注意LIS要求你求单调递增还是不上降, 求以 a_i 开头的LIS长度是多少可以用倒叙加 `greater<>`; 要求你单调增的LIS需要用`lowerbound`
- **关键代码:**

```
void solve()
{
    int n;
    cin >> n;
    vi nm(n);
    rep(i, 0, n - 1)
    {
        cin >> nm[i];
    }

    auto LIS = [&](vi &aa) -> void
    {
        vector<int> d;
        for (int i = 0; i < n; i++)
        {
            if (d.empty() || nm[i] > d.back())
            {
                d.push_back(nm[i]);
                aa[i] = d.size();
            }
            else
            {
                auto it = lower_bound(all(d), nm[i]);
                *it = nm[i];
                aa[i] = it - d.begin() + 1;
            }
            // cerr << aa[i] << ' ' << i << '\n';
        }
    };

    auto LISx = [&](vi &aa) -> void
    {
        vector<int> d;
        for (int i = n - 1; i >= 0; i--)
        {
            if (d.empty() || nm[i] < d.back())
            {
                d.push_back(nm[i]);
                aa[i] = d.size();
            }
            else
            {
                auto it = lower_bound(all(d), nm[i], greater<>());
                *it = nm[i];
                aa[i] = it - d.begin() + 1;
            }
            // cerr << aa[i] << ' ' << i << '\n';
        }
    };
}
```

```

    }
};
vi pre(n);
LIS(pre);
vi sub(n);
LISx(sub);

map<int, int> proans3;
int mx = *max_element(all(pre));
//cerr << mx << '\n';
for (int i = 0; i < n; i++)
{
    if (pre[i] + sub[i] - 1 == mx)
    {
        // cerr<<11<<'\n';
        proans3[pre[i]]++;
    }
}
for (int i = 0; i < n; i++)
{
    if (pre[i] + sub[i] - 1 != mx)
    {
        cout << 1;
    }
    else
    {
        if (proans3[pre[i]] == 1)
        {
            cout << 3;
        }
        else
            cout << 2;
    }
}
}
}

```

F. Pizza Delivery F. 披萨配送

- **核心模型**:滚动数组优化dp
- **思维误区 (Bug)**:状态转移
- **修正逻辑 (Patch)**:处理边界条件于状态转移: 起点到终点的距离+len
- **关键代码**:

```

void solve()
{
    int n, sx, sy, ex, ey;
    cin >> n >> sx >> sy >> ex >> ey;
    vector<pii> pt(n);
    set<int> cnt;
    for (int i = 0; i < n; i++)

```

```

{
    cin >> pt[i].first;
    cnt.emplace(pt[i].first);
}
for (int i = 0; i < n; i++)
{
    cin >> pt[i].second;
}
int lien = cnt.size();
sort(all(pt), [](pii a, pii b)
    { return a.first < b.first; });

ll ans = 0;
vector<vi> lie(lien);

vi dx(lien + 1);
dx[0] = pt[0].first - sx;
dx[lien] = pt[n - 1].first - ex;

int tp = 0;
for (int i = 0; i < n; i++)
{
    if ((i != 0 && pt[i].first != pt[i - 1].first))
    {
        tp++;
        dx[tp] = pt[i].first - pt[i - 1].first;
    }
    lie[tp].push_back(pt[i].second);
}

vi up(lien);
vi down(lien, 1e9); // 错误初始化
for (int i = 0; i < lien; i++)
{
    for (int j = 0; j < lie[i].size(); j++)
    {
        up[i] = max(up[i], lie[i][j]);
        down[i] = min(down[i], lie[i][j]);
    }
}

int dpl = 0;
int dpr = 0;

for (int i = 0; i < lien; i++)
{
    ll len = up[i] - down[i];

    int prel = dpl;
    int prer = dpr;
    if (!i)
    {
        dpl = dx[i] + abs(sy - up[0]) + len;
        dpr = dx[i] + abs(sy - down[0]) + len;
    }
}

```



```

    }
    else
    {
        dpr = dx[i] + min(abs(down[i - 1] - down[i]) + prel, abs(up[i - 1] -
down[i]) + prer) + len;
        dpl = dx[i] + min(abs(down[i - 1] - up[i]) + prel, abs(up[i - 1] -
up[i]) + prer) + len;
    }
}
ll last_d = abs(dx[lien]);

ans = min(
    dpl + abs(down[lien - 1] - ey), // 最后停在下端点 -> 终点
    dpr + abs(up[lien - 1] - ey) // 最后停在上端点 -> 终点
) +
last_d;

cout << ans << "\n";
}

```

最大子段和

```

>
> P1115 最大子段和
>
> 题目描述
>
> 给一个长度为  $n$  的序列  $a$ ，选出其中连续且非空的一段使得这段和最大。
>
> 输入格式
>
> 第一行是一个整数，表示序列的长度  $n$ 。
> 第二行有  $n$  个整数，第  $i$  个整数表示序列的第  $i$  个数字  $a_i$ 。
>
> 输出格式
>
> 输出一行一个整数表示答案。
>
> 数据规模与约定
>
> - 对于  $40\%$  的数据，保证  $n \leq 2 \times 10^3$ 。
> - 对于  $100\%$  的数据，保证  $1 \leq n \leq 2 \times 10^5$ ， $-10^4 \leq a_i \leq 10^4$ 。

```

- **AC代码**

```
```cpp
```

```

void solve()
{
 int n;
 cin >> n;

```

```

vi nums(n+1);

for (int i=1;i<=n;i++) cin>>nums[i];
vi dp(n+1);
int ans=-4545556565;
for(int j=1;j<=n;j++)
{
 dp[j]=dp[j-1]+nums[j];
 if (dp[j]<=0) dp[j]=nums[j];
 ans=max(ans,dp[j]);
}
cout<<ans<<'\n';
}

```

## A. Against the Difference

### • 题目

We define that a block is an array where all elements in it are equal to the length of the array. For example,  $[3, 3, 3]$ ,  $[1]$ , and  $[4, 4, 4, 4]$  are blocks, while  $[1, 1, 1]$  and  $[2, 3, 3]$  are not. An array is called neat if it can be obtained by the concatenation of an arbitrary number of blocks (possibly zero). Note that an empty array is always neat. You are given an array  $a$  consisting of  $n$  integers. Find the length of its longest neat subsequence\*. \*A sequence  $c$  is a subsequence of a sequence  $a$  if  $c$  can be obtained from  $a$  by the deletion of several (possibly, zero or all) element from arbitrary positions. 我们定义block是一个数组，其中的所有元素都等于该数组的长度。例如， $[3, 3, 3]$ ， $[1]$ 和 $[4, 4, 4, 4]$ 是块，而 $[1, 1, 1]$ 和 $[2, 3, 3]$ 不是块。如果数组可以通过任意数量的块（可能为零）的连接获得，则称为整齐数组。注意，空数组总是整洁的。给定一个由  $n$  个整数组成的数组  $a$ 。求它的最长整洁子序列\*的长度。\*序列  $c$  是序列  $a$  的子序列，如果  $c$  可以从  $a$  中删除任意位置的几个（可能为零或全部）元素。

### • 思路

- 首先这道题目是典型dp，因为他是通过删掉一些数字找一个最大的子序列，对于每个数字来说他面临两种状态：删掉或者不删掉（或者说是选择或者不选择）。而决定他是否被选择的权值就是到目前这个位置他做多少做贡献。容易发现只有在他结算的时候才会做贡献。我们只需要贪心地（我的意思是他要找离他最近地跟他数字相同的，这样子才能给后面留空间。因为递推所以有  $i < j$  必然有  $dp[i] \leq dp[j]$  找他开头的那一时刻的那一时刻结算就行。

- AC代码：**用桶和打表数组来处理每个数字的位置

```

void solve()
{
 int n;
 cin>>n;
 vi nums(n+1);
 vector<vi> idx(n+1);
 vi ery(n+1);
 rep(i,1,n)
 {

```

```

 cin>>nums[i];
 idx[nums[i]].push_back(i);
 eryl[i]=idx[nums[i]].size();
 }

 vi dp(n+1);
 dp[0]=0;
 vi xz(n+1);
 for(int i=1;i<=n;i++)
 {
 int pans=0;
 int cur=eryl[i]-nums[i];
 if(cur>=xz[i]*nums[i])
 {
 pans=dp[idx[nums[i]]][cur]-1+nums[i];
 xz[nums[i]]++;
 }
 dp[i]=max(pans,dp[i-1]);
 }

 cout<<dp[n]<<'\n';
}

```

### \*G. Mukhammadali and the Smooth Array

非常有意思的题目，请严肃学习1e9次题解

- 题号: G
- 链接: [题目链接](#)
- 算法类型: DP 题目拆解与优化
- 错误原因:
  - 贪心局部决策与回退混乱，无法考虑全局最优解
  - 贪心只看“能不能接”，没看“值不值”
- AC 代码 做法一：逆向加权LIS:

```

void solve()
{
 int n;
 cin >> n;
 vi a(n + 1);
 vi c(n + 1);
 for (int i = 1; i <= n; i++)
 {
 int d;
 cin >> d;
 a[i] = d;
 }
}

```

```

int sumc=0;
for (int i = 1; i <= n; i++)
{
 cin >> c[i];
 sumc+=c[i];
}
if (n == 1)
{
 cout << "0\n";
 return;
}
vi dp(n+1);
for(int i=1;i<=n;i++)
{
 dp[i]=c[i];
 for(int j=1;j<i;j++)
 {
 if(a[j]<=a[i])
 {
 dp[i]=max(dp[i],dp[j]+c[i]);
 }
 }
}
int hsh=*max_element(dp.begin()+1,dp.end());
cout<<sumc-hsh<<'\n';
}

```

### • 做法解析:

- 逆向思维：将修改价值和最小 -> 一定不用修改价值和最大

首先我们可以知道,如果贪心修改会发现你对这个值 $a[x]$ 得修改会影响后面数列大小关系的全局,就不满足DP里面最有子解以及有向无环的改变策略。对于每个需要修改的 $a[x]$ 你需要判断其修改后一定不影响后面解集。! 但是! 如果你 $a[i+1]$ 改大了,可能后面开销会大,如果你 $a[i]$ 改小了,可能反过来不满足前面的序列增减 **决策相互耦合 → 形成环 → 无法 DP**

事实上正向也能做,但是不是贪心

- 逆向思维==>当子集为LIS时这几个位置是一定不用修改的

这时候会有个疑问: 对于没有加进LIS队伍的的数字是否一定保证他们有一个解且这个解不会影响LIS队伍中元素的修改? 显而易见是一定的。问题二: 对于没有加进lis的队伍有没有可能因为别人被修改过从而更优剩下了自己修改的开销? 对于 LIS 他是'LIS 资格证', 而不是'LIS 全家桶'。将原数组的 LIS 算完之后剩下的点都是离散的 **更加规范化的表达**: 我们求出的  $\max(dp[i])$  对应的非降子序列, 是一个局部最优解 (以某个  $i$  结尾), 但它不一定包含所有可保留点。未被选中的点是『可独立修改』的, 因为它们不参与最优链

### • 思路:

- 最小修改成本, → 最大保留收益, 本题
- 最小删除次数, → 最长上升子序列, LIS
- 最小交换次数, → 逆序对计数, 冒泡排序

- 为何最优解：当问题从寻找如何修改变成寻找lis时（带权重的LIS），我们知道接下来要最大化LIS权和，然后发现有重叠子问题（其实lis就是非常适合dp的），直接dp； ==所有单调子序列问题，本质都是 DAG 上的最长路径。 == ==DP 转移 = 枚举前驱， max = 最长路径。 ==
- **AC 代码** 做法二 顺序dp:

```
void solve()
{
 int ans = 1e18;
 int n;
 cin >> n;
 vector<int> a(n);
 vector<int> c(n);
 for (int i = 0; i < n; i++)
 {
 cin >> a[i];
 }

 for (int i = 0; i < n; i++)
 {
 cin >> c[i];
 }

 vector<int> cpy = a;
 sort(cpy.begin(), cpy.end());
 int m = unique(cpy.begin(), cpy.end()) - cpy.begin();

 vector<int> dp(m);
 vector<int> mn(m);
 int cmn = 1e18;
 cpy.resize(m);
 for (int i = 0; i < m; i++)
 {
 if (a[0] == cpy[i])
 {
 dp[i] = 0;
 }
 else
 {
 dp[i] = c[0];
 }
 cmn = min(cmn, dp[i]);
 mn[i] = cmn;
 }

 for (int i = 1; i < n; i++)
 {
 cmn = 1e18;
 vector<int> cpy(m);
 for (int j = 0; j < m; j++)
 {
 int tp;
 if (a[i] == cpy[j])
```

```

 {
 tp = 0;
 }
 else
 {
 tp = c[i];
 }
 tp+=mn[j];
 dp[j] = tp;
 //mn[i] = cmn;
 cmn = min(cmn,dp[j]);
 cpn[j] = cmn;
 }
 mn = cpn;
}

cout << mn[m-1] << '\n';
//cout << flush;
}

```

#### • 表达方式学习:

- `sort(cpy.begin(), cpy.end()); int m = unique(cpy.begin(), cpy.end()) - cpy.begin();` 这一步是为cpy去重unique函数。然后获得去重后容器的大小（不能用size!! **unique 不改变容器大小!**）。记得unique前==先排序==
- `cpy.resize(m)`重新变更容器大小

## 位运算DP

### P4310 绝世好题

#### 不辜负他的题目名字，确实是绝世好题

#### • 题目

给定一个长度为  $n$  的数列  $a_i$ ，求  $a_i$  的子序列  $b_i$  的最长长度  $k$ ，满足  $b_i \& b_{i-1} \neq 0$ ，其中  $2 \leq i \leq k$ ， $\&$  表示位运算取与。

- **解析**：有脑子的人一眼就能写出dp方程，解法类似于求LIS，但是这是n方做法，本题数据范围1e5。必然要想出一种nlogn级别的优化，怎么优化呢？实现上这是一种位运算特有的思路:每一个位置拆开来进dp

#### • AC代码

```

vi dp(32,0);
int zuida=-1;
for (int hsh : nums)
{
 for (int i = 0; i < 32; i++)
 {
 if ((hsh >> i) & 1) {

```

```

 zuida=max(dp[i]+1,zuida);
 }
}
for (int i = 0; i < 32; i++)
{
 if ((hsh >> i) & 1) {
 dp[i]=zuida;
 }
}
}

cout<<zuida;

```

## 区间dp

- 目前我做到两种区间dp：
  - 一种是整一块是由两个小块拼接版本的
    - 这对应的是树形结构的合并，根节点由左右两个子树组成。
    - $$dp[i][j] = \max_{k=i}^{j-1} dp[i][k] \oplus dp[k+1][j]$$
  - 一种是类似于洋葱剥皮的递推版本
    - 这对应的是线性结构的延伸，只在两头做文章。
    - $$dp[i][j] = dp[i+1][j] \vee dp[i][j-1] \vee (dp[i+1][j-1] + \text{cost})$$

## P1435 [IOI 2000] 回文字串

- **核心模型**:一种是类似于洋葱剥皮的递推版本
- **思维误区 (Bug)**:关键初始化
- **修正逻辑 (Patch)**:回文的定义是什么？是对称。对称这个性质，是由两端决定的，而不是由中间决定的。此处默认dp起点是必定回文
- **关键代码**:

```

void solve()
{
 string s;
 cin >> s;
 int n = s.size();
 vector<vi> dp(n + 1, vi(n + 1, INF));

 for (int i = 0; i < n - 1; i++)
 {
 if (s[i] == s[i + 1])
 {
 dp[i][i + 1] = 0;

```

```

 }
}

for (int len = 1; len <= n; len++)
{
 for (int l = 0; l + len - 1 < n; l++)
 {
 int r = l + len - 1;
 if (r == l)
 {
 dp[l][r] = 0;
 }
 else
 {
 if (s[l] == s[r])
 {
 dp[l][r] = min(dp[l][r], dp[l + 1][r - 1]);
 }

 dp[l][r] = min(dp[l][r], min(dp[l + 1][r] + 1, dp[l][r - 1] + 1));
 }
 }
}

cout << dp[0][n - 1];
}

```

### P4290 [HAOI2008] 玩具取名

- **核心模型**:一种是整一块是由两个小块拼接版本的
- **思维误区 (Bug)**:因为题目规则是 二合一 ( $A \rightarrow BC$ )。这本质上是一棵二叉树的构建过程。当你合并  $A(BCD)$ 可能会遗漏掉 $(AB)(CD)$
- **修正逻辑 (Patch)**:
- **关键代码**:

### 典题P3146 [USACO16OPEN]248

- **核心模型**:区间dp, **看到合并一类的操作**; 区间 DP 的核心逻辑就是“大区间由小区间合并而来”。只要涉及到“合并”或者“拆分”, 你都得知道从哪儿合并, 也就是要找那个  $k$ 。
- **思维误区 (Bug)**:记得区间写左闭右闭
- **修正逻辑 (Patch)**:
- **关键代码**:



```

void solve()
{
 int n;
 cin >> n;
 vector<vi> dp(n, vi(n, -1));
 ll ans = 0;
 rep(i, 0, n - 1)
 {
 cin >> dp[i][i];
 ans = max(ans, (ll) dp[i][i]);
 }
 // 枚举长度, 枚举起点, 枚举断点

 for (int len = 2; len <= n; len++)
 {
 for (int i = 0; i + len - 1 < n; i++)
 {
 for (int k = i; k < i + len - 1; k++)
 {
 if (dp[i][k] == dp[k + 1][i + len - 1] && dp[i][k] > 0)
 {
 dp[i][i + len - 1] = max(dp[i][k] + 1, dp[i][i + len - 1]);
 ans = max(ans, (ll)dp[i][i + len - 1]);
 }
 }
 }
 }

 cout << ans;
}

```

### P3147 [USACO16OPEN] 262144 P(待补题)

- **核心模型:** 数据范围:  $2 \leq N \leq 262,144$  , 每个数的范围在  $1 \dots 40$  之间
- **思维误区 (Bug):**
- **修正逻辑 (Patch):**
- **关键代码:**

```

void solve()
{
 int n;
 cin >> n;
 // vi nums(n);
 ll ans = 0;
 vector<vi> dp(61, vi(n + 3, -1));
 for (int i = 0; i < n; i++)
 {
 int in = 0;
 cin >> in;
 dp[in][i] = i + 1;
 }
}

```

```

 ans = max(ans, (ll)in);
 }

 for (int i = 1; i <= 60; i++)
 {
 for (int j = 0; j < n; j++)
 {
 if (dp[i][j] == -1)
 {
 dp[i][j] = dp[i - 1][dp[i - 1][j] + 1];
 }
 if (dp[i][j] != -1)
 {
 // dp[i][j]=dp[i][dp[i-1][j]+1];
 // dp[i][j]=dp[i][dp[i-1][j]+1];
 ans = max((ll)i, ans);
 }
 }
 }

 cout << ans;
}

```

### P1070 道路游戏 (处理环形)

- **核心模型:**
- **环形处理:** 1-based
  - 往后走 (顺时针):  $\text{next\_pos} = (x + k - 1) \% n + 1$
  - 往前走 (逆时针退后):  $\text{prev\_pos} = (x - k - 1 + n) \% n + 1$   $\text{prev\_pos} = ((x - k - 1) \% n + n) \% n + 1$
  - **0based** 环形公式极其简单, 往后走就是  $(x + k) \% n$ , 往前退就是  $(x - k + n) \% n$ 。
- **修正逻辑 (Patch):**
- **关键代码:**

### 魔法项链 (o1简单查找连乘思路, 断环成链教学)

- **核心模型:** 区间dp
- **思维误区 (Bug):** 注意这里项链的合并逻辑, 我们dp数组里面存的是释放能量。根据题意
- **修正逻辑 (Patch):** 注意到段换成链: 可以on去最大值 (就不用在循环里写可能错) 以及成链的做法
- **关键代码:**

```

void solve()
{

```

```

int n;
cin >> n;
vector<vi> dp(2 * n + 4, vi(2 * n + 4, 0));
vi x1(2 * n + 3);
rep(i, 1, n)
{
 cin >> x1[i];
 x1[i + n] = x1[i];
}

for (int len = 2; len <= n; len++)
{
 for (int l = 1; l + len <= 2 * n; l++)
 {
 int r = l + len;

 for (int k = l + 1; k < r; k++)
 {
 dp[l][r] = max(dp[l][r], dp[l][k] + dp[k][r] + (x1[l] * x1[k] *
x1[r]));
 }
 }
}
int ans = 0;
for (int i = 1; i <= n; i++)
{
 ans = max(ans, dp[i][i + n]);
}
cout << ans;
}

```

## 关路灯

- **核心模型:**区间dp
- **思维误区 (Bug):**前缀和维护“剩下没关的灯”
- **修正逻辑 (Patch):**递推关系
- **关键代码:**

```

struct deng
{
 int spot, wat;
 int qzh = 0;
};
void solve()
{
 int n, c;
 cin >> n >> c;
 vector<deng> lt(n + 4);
 vector<vector<vi>> dp1(n + 2, vector<vi>(n + 2, vi(2, INF)));
 vector<vi> dp2(n + 2, vi(n + 2, 0));
}

```

```

rep(i, 1, n)
{
 cin >> lt[i].spot >> lt[i].wat;
 lt[i].qzh = lt[i].wat + lt[i - 1].qzh;
}

dp1[c][c][0] = 0;
dp1[c][c][1] = 0;
for (int len = 2; len <= n; len++)
{
 for (int l = 1; l + len - 1 <= n; l++)
 {
 int r = l + len - 1;
 if (l <= c && r >= c)
 {
 dp1[l][r][0] = min(dp1[l][r][0], min(dp1[l + 1][r][1] +
 (abs(lt[r].spot - lt[l].spot) * (-lt[r].qzh + lt[l].qzh + lt[n].qzh)),
 dp1[l + 1][r][0] + (abs(lt[l
+ 1].spot - lt[l].spot) * (-lt[r].qzh + lt[l].qzh + lt[n].qzh)))));
 dp1[l][r][1] = min(dp1[l][r][1], min(dp1[l][r - 1][1] +
 (abs(lt[r].spot - lt[r - 1].spot) * (-lt[r - 1].qzh + lt[l - 1].qzh + lt[n].qzh)),
 dp1[l][r - 1][0] +
 (abs(lt[r].spot - lt[l].spot) * (-lt[r - 1].qzh + lt[l - 1].qzh + lt[n].qzh)))));
 //简单点说，它就是在算：除了你已经搞定的那一块区域，剩下的灯加起来一共多
 亮（多耗电）。
 }
 }
}
// cout << dp1[1][n][0] << ' ' << dp1[1][n][1];
cout << min(dp1[1][n][0], dp1[1][n][1]);
}

```

### SUE的小球（跟关灯完全相同）

- **核心模型:**
- **思维误区 (Bug):** 数组开大导致MLE，答案数字比较大需要初始化为LLINF。注意到必须尽量在魅力值高的时候收集这个彩蛋，而如果一个彩蛋掉入海中，它的魅力值将会变成一个负数，但这并不影响 Sue 兴趣。注意到我们无法贪心的去找到在这段上面消耗的时间（因为有可能往左或者往右，因为是区间dp）。所以我们仅能逆向思路：求“损失最小值”。处理方法前缀和维护“彩球失去的价值”。
- **修正逻辑 (Patch):**
- **关键代码:** 注释掉的逻辑是假的。

```

struct pt
{
 int x, y, v;
};

void solve()
{

```

```

int n, x0;
cin >> n >> x0;
vector<pt> ptt(n + 3);
for (int i = 0; i < n; i++)
{
 cin >> ptt[i].x;
}
ll sum = 0;
for (int i = 0; i < n; i++)
{
 cin >> ptt[i].y;
 sum += ptt[i].y;
}

// 求段lr总和为qzh[r+1]-qzh[l]
// 求剩下段总和为(qzh[n]-qzh[0]-(qzh[r+1]-qzh[l]))
// 加上距离 (时间) (ptt[])

rep(i, 0, n - 1) cin >> ptt[i].v;
ptt.push_back({x0, 0, 0});
n = ptt.size();
sort(all(ptt), [](pt a, pt b)
 { return a.x < b.x; });
vi qzh(n + 1);
for (int i = 0; i < n; i++)
{
 qzh[i + 1] = qzh[i] + ptt[i].v;
}

for (int i = 0; i < n; i++)
{
 if (ptt[i].x == x0)
 {
 x0 = i;
 break;
 }
}

//-----//
// 希望是给个时间返回彩蛋分数
// auto cd = [&](int timm, int i_x)
// {
// return ptt[i_x].y - (ptt[i_x].v * timm);
// };

// auto when = [&](int l, int r, int sp)
// {
// if (sp == 0)
// {
// // return ptt[r].x - ptt[l].x + (ptt[r].x - ptt[x0].x - 1);
// return (ptt[r].x - ptt[l].x) + (ptt[r].x - ptt[x0].x);
// }
// else
// {

```

```

// // return ptt[r].x - ptt[l].x + (ptt[x0].x - ptt[l].x - 1);
// return (ptt[r].x - ptt[l].x) + (ptt[x0].x - ptt[l].x);
// }
// };

//-----//
vector<vector<vi>> dp(n + 5, vector<vi>(n + 5, vi(2, LINF)));

dp[x0][x0][0]=0;
dp[x0][x0][1]=0;
for (int len = 2; len <= n; len++)
{
 for (int l = 0; l + len - 1 < n; l++)
 {
 int r = l + len - 1;
 if (l <= x0 && r >= x0)
 {
 //-----//
 // dp[l][r][0] = max(dp[l + 1][r][0] + cd(when(l + 1, r, 0) +
 ptt[l + 1].x - ptt[l].x, l), dp[l + 1][r][1] + cd(when(l + 1, r, 1) + ptt[l + 1].x
 - ptt[l].x, l));
 // dp[l][r][1] = max(dp[l][r - 1][0] + cd(when(l, r - 1, 0) +
 ptt[r].x - ptt[r - 1].x, r), dp[l][r - 1][1] + cd(when(l, r - 1, 1) + ptt[r].x -
 ptt[r - 1].x, r));
 // dbg(l, r);
 //-----//
 dp[l][r][0] = min(dp[l + 1][r][0] + (ptt[l + 1].x - ptt[l].x) *
 (qzh[n] - (qzh[r + 1] - qzh[l + 1])),
 dp[l + 1][r][1] + (ptt[r].x - ptt[l].x) *
 (qzh[n] - (qzh[r + 1] - qzh[l + 1])));
 dp[l][r][1] = min(dp[l][r - 1][0] + (ptt[r].x - ptt[l].x) *
 (qzh[n] - (qzh[r] - qzh[l])),
 dp[l][r - 1][1] + (ptt[r].x - ptt[r - 1].x) *
 (qzh[n] - (qzh[r] - qzh[l])));
 }
 }
}

double ans = sum - min(dp[0][n - 1][0], dp[0][n - 1][1]);
ans /= 1000;
cout << fixed << setprecision(3) << ans;
}

```

## 石子合并 (包括四边形优化)

- **核心模型**: 这里是四边形优化教学
- **四边形优化**:
  - 1. 适用方程的形式 (标准型)
    - 首先, 你的状态转移方程必须是区间 DP 的经典形式:

$$dp[i][j] = \min_{i \leq k < j} dp[i][k] + dp[k+1][j] + w(i, j)$$

- 其中:  $dp[i][j]$  表示合并区间  $[i, j]$  的最优代价。 $w(i, j)$  是合并这一步产生的直接代价 (例如石子合并里的区间和)。
- 目的是求 最小值。
- 1. 代价函数  $w(i, j)$  需满足两个数学性质要能优化, 核心不在  $dp$  数组本身, 而在于那个代价函数  $w(i, j)$ 。
- 它必须满足: A. 区间包含单调性 (Monotonicity) 含义: 小区间的代价  $\leq$  包含它的大区间的代价。
  - 数学表达: 对于任意  $i \leq i' < j' \leq j$ , 有:

$$w(i', j') \leq w(i, j)$$

B.

- 四边形不等式 (Quadrangle Inequality) 含义: 交叉小于包含。即“两个交错区间的代价和”小于等于“它们并集区间与交集区间的代价和”。数学表达: 对于任意  $i < i' < j < j'$ , 有:

$$w(i, j) + w(i', j') \leq w(i, j') + w(i', j)$$

- 严肃注意: 四边形优化有单调性所以只能优化求最小值:
- 严肃注意: 四边形优化  $dp$  数组要开大一点,  $solu$  数组要开大一点, 以及初始化: 初始化 `solu[i][i]`
- 注意: 断环成链的处理最好开多几个
- 关键代码:

```
void solve()
{
 int n;
 cin >> n;
 vi nums(2 * n);

 vector<vi> dp(2 * n + 2, vi(2 * n + 2, 0));
 vector<vi> dp2(2 * n + 2, vi(2 * n + 2, 0));
 vi qzh(2 * n + 1, 0);
 vector<vi> solu(2 * n + 5, vi(2 * n + 5, 0));
 for (int i = 0; i < n; i++)
 {
 cin >> nums[i];
 nums[i + n] = nums[i];
 }
 for (int i = 0; i < 2 * n; i++)
 {
 qzh[i + 1] = qzh[i] + nums[i];
 solu[i][i] = i;
 }

 for (int len = 2; len <= n; len++)
 {
 for (int l = 0; l + len - 1 < 2 * n; l++)
 {
 dp2[l][l + len - 1] = max(dp2[l + 1][l + len - 1], dp2[l][l + len - 2]) + qzh[l + len] - qzh[l];
```

```

 dp[l][l + len - 1] = INF;
 for (int k = solu[l][l + len - 2]; k <= solu[l + 1][l + len - 1]; k++)
 {
 int val = dp[l][k] + dp[k + 1][l + len - 1] + (qzh[l + len] -
qzh[l]);
 if (dp[l][l + len - 1] > val)
 {
 dp[l][l + len - 1] = val;
 solu[l][l + len - 1] = k; // 记录决策点, 供下一轮长度使用
 }
 }
 }
 int mn = INF, mx = 0;

 for (int i = 0; i < n; i++)
 {
 mn = min(mn, dp[i][i + n - 1]);
 mx = max(mx, dp2[i][i + n - 1]);
 }

 cout << mn << '\n'
 << mx << '\n';
}

```

## 01 背包 (一维/二维)

**用途:** 每个物品选或不选, 求最大价值 (不要求装满/要求恰好装满)。

**常见坑:**

- 一维数组需倒序循环, 防止覆盖。
- 恰好装满需初始化  $dp[0]=0$ ,  $dp[1...v]=-INF$ 。
- 注意 long long 防溢出。

### 一维数组

```

void solve() {
 int n, v;
 cin >> n >> v;
 struct kind { int value, weight; };
 vector<kind> nums(n);
 for (int i = 0; i < n; i++) {
 cin >> nums[i].weight >> nums[i].value;
 }
 vector<int> dp(v + 1);
 vector<int> dp2(v + 1, -99999999); // 恰好装满
 dp[0] = 0;
 for (int i = 0; i < n; i++) {
 for (int j = v; j >= nums[i].weight; j--) {

```



```

 dp[j] = max(dp[j], dp[j - nums[i].weight] + nums[i].value);
 dp2[j] = max(dp2[j], dp2[j - nums[i].weight] + nums[i].value);
 }
}
cout << dp[v] << '\n';
cout << (dp2[v] > 0 ? dp2[v] : 0) << '\n';
}

```

## 二维数组

```

void solve() {
 int t, m; // t=容量, m=物品数
 cin >> t >> m;
 struct cy { int time, ac; };
 vector<cy> cord(m + 1);
 for (int i = 1; i <= m; i++) {
 cin >> cord[i].time >> cord[i].ac;
 }
 int dp[105][1005] = {};
 for (int i = 1; i <= m; i++) {
 for (int j = 0; j <= t; j++) {
 if (j >= cord[i].time)
 dp[i][j] = max(dp[i-1][j-cord[i].time] + cord[i].ac, dp[i-1][j]);
 else
 dp[i][j] = dp[i-1][j];
 }
 }
 cout << dp[m][t] << '\n';
}

```

### 复杂度:

- 一维:  $O(n*v)$ , 空间  $O(v)$ 。
- 二维:  $O(nt)$ , 空间  $O(nt)$ 。

## 完全背包

**用途:** 物品可无限选, 求最大价值。

**常见坑:** 正序循环 (区别于 01 背包倒序), 防止重复选择。

```

void solve() {
 int time, n;
 cin >> time >> n;
 struct cy { int t, v; };
 vector<cy> hsh(n + 1);
 for (int i = 1; i <= n; i++)
 cin >> hsh[i].t >> hsh[i].v;
 vector<int> dp(time + 1);
 for (int i = 1; i <= n; i++) {

```

```

 for (int j = hsh[i].t; j <= time; j++)
 dp[j] = max(dp[j - hsh[i].t] + hsh[i].v, dp[j]);
 }
 cout << dp[time] << '\n';
}

```

**复杂度:**  $O(n \cdot \text{time})$ , 空间  $O(\text{time})$ 。

## 分组背包(KN)

### 漫步大地的游医

- **核心模型:**多重背包
- **思维误区 (Bug):**算错复杂度; 状态顺序有问题
- **修正逻辑 (Patch):**对于多重背包, 选择数量跑在外面->拿了一个再拿两个, 两个事件之间是独立的x 我们需要先美剧体积再枚举决策, 对于特定的体积尝试
- **关键代码:**

```

for (int i = 0; i < p; i++)
{
 for (int q = k; q >= 0; q--)
 {
 for (int j = 0; j <= cnt[i]; j++)
 {
 if (q - j >= 0)
 dp[q] = max(dp[q], dp[q - j] + (int)log2(1 + j) * gx[i]);
 }
 }
}

```

## 其他dp

### 切蛋糕

- **错误原因:**没有错, 单纯想要记录天才异或dp
- **题解思路:**

建立二维数组, 直接模拟遍历切蛋糕, 通过队每个  $(i, j)$  进行异或运算来记录每个1,  $1 \rightarrow i, j$  的可行状态。由几何关系可知每个  $(i, j)$  的状态仅仅取决于  $(i-1, j)$  和  $(i, j-1)$ 。虽然这道题很简单我的解法比dp更简单更省空间

我想说的是: 由一个状态继承过来的做法就可以叫做dp